

# Learning AI Training Guide

The Learning AI uses Deep Q-Learning (DQN) with PyTorch to learn optimal gin rummy play through self-play and curriculum training.

## Quick Start

```
# Install learning dependencies
```

```
uv sync --extra learning
```

```
# Train a model (default: 10,000 episodes)
```

```
uv run gin-train --episodes 10000 --output models/learning_ai.pt
```

```
# Use trained model in simulation
```

```
uv run gin-simulate --ai1-type learning --ai1-model models/learning_ai.pt --ai2-type context -n 100
```

## Architecture

### Three Separate Networks

The AI uses three specialized neural networks, one for each decision type:

| Network    | Decision              | Input Size | Output      | Hidden Layers |
|------------|-----------------------|------------|-------------|---------------|
| DrawNet    | Deck vs discard       | 198        | 2 Q-values  | [128, 64]     |
| DiscardNet | Which card to discard | 250        | 52 Q-values | [256, 128]    |
| KnockNet   | Whether to knock      | 198        | 2 Q-values  | [64, 32]      |

Note: DiscardNet outputs 52 Q-values (one per card in deck). During inference, cards not in hand are masked with `-inf` so only valid discards are selected. This allows the network to learn card-specific values rather than position-based values.

### State Representation (~200 features)

The game state is encoded as a fixed-size tensor:

State Vector Components:

```
+++ Hand (52)          - One-hot: which cards are in hand
+++ Dead cards (52)     - Multi-hot: cards known unavailable
+++ Discard top (52)    - One-hot: top card of discard pile
+++ Opponent patterns:
|  +++ Pickup ranks (13) - Normalized counts
|  +++ Pickup suits (4)  - Normalized counts
|  +++ Discard ranks (13) - Normalized counts
|  +++ Discard suits (4) - Normalized counts
+++ Game features (8):
|  +++ deck_position_pct - 0.0 (full) to 1.0 (empty)
|  +++ my_deadwood_norm  - deadwood / 100
|  +++ can_knock         - 1.0 if deadwood <= 10
|  +++ is_gin            - 1.0 if deadwood == 0
|  +++ score_diff_norm   - (my_score - opp_score) / 100
|  +++ points_to_win_norm - points_to_win / 100
|  +++ meld_count_norm   - num_melds / 4
|  +++ live_outs_norm    - live_outs / 20
+++ Drawn card (52)     - One-hot (discard decision only)
```

## Training Process

### Deep Q-Learning (DQN)

The training uses standard DQN with:

1. **Experience Replay:** Stores (state, action, reward, next\_state, done) tuples in a buffer, samples random batches for training to break correlation between consecutive experiences.
2. **Target Network:** A separate copy of the networks updated periodically (every 100 episodes) to stabilize training.
3. **Epsilon-Greedy Exploration:** Starts with 100% random actions, decays to 5% over training.

### Reward Function

**End of Round Rewards:** | Outcome | Reward | |———|———| | Win by gin | +50 | | Win by knock | +20 + points/5 | | Win by undercut | +30 + points/5 | | Loss | -points/5 | | Draw | 0 |

**Intermediate Rewards (per turn):** - Deadwood reduction: +0.1 per point reduced - Meld completion: +1.0 per new meld - Key out pickup: +0.5 for meld-completing cards

### Curriculum Learning

Training progresses through stages with increasingly difficult opponents:

| Stage | Opponent       | Episodes | Purpose             |
|-------|----------------|----------|---------------------|
| 1     | BasicAI        | 5,000    | Learn fundamentals  |
| 2     | ContextAwareAI | 5,000    | Learn advanced play |
| 3     | Self-play      | 10,000   | Refine strategy     |

## CLI Options

### gin-train

```
uv run gin-train [OPTIONS]
```

Options:

```
--episodes N      Number of training episodes (default: 10000)
--output PATH     Path to save model (default: models/learning_ai.pt)
--tensorboard DIR TensorBoard log directory (default: runs/gin_learning)
--resume PATH     Resume training from checkpoint
--eval-freq N     Evaluate every N episodes (default: 500)
--save-freq N     Save checkpoint every N episodes (default: 1000)
-v, --verbose     Enable verbose logging
```

### gin-simulate with LearningAI

```
uv run gin-simulate \
  --ai1-type learning \
  --ai1-model models/learning_ai.pt \
  --ai2-type context \
  -n 100
```

## Hyperparameter Experiments

Use gin-experiment for easy A/B testing of different configurations.

## Quick Start

```
# List available presets
uv run gin-experiment --list-presets

# Use a preset
uv run gin-experiment --preset fast           # 1000 episodes, quick test
uv run gin-experiment --preset low-lr        # lr=0.0003
uv run gin-experiment --preset aggressive-rewards # Higher reward signals

# Custom hyperparameters
uv run gin-experiment --lr 0.0003 --batch-size 128 --episodes 10000

# Named experiment (for easy comparison)
uv run gin-experiment --name "test_v2" --lr 0.0003 --gamma 0.95
```

## Available Presets

| Preset             | Description              | Key Changes      |
|--------------------|--------------------------|------------------|
| fast               | Quick test run           | 1000 episodes    |
| standard           | Standard training        | 10000 episodes   |
| thorough           | Thorough training        | 25000 episodes   |
| low-lr             | Lower learning rate      | lr=0.0003        |
| big-batch          | Larger batch size        | batch_size=128   |
| slow-explore       | Slower exploration decay | decay=0.9998     |
| aggressive-rewards | Higher reward signals    | gin=75, knock=30 |

## All Hyperparameter Options

```
uv run gin-experiment [OPTIONS]
```

### Training Parameters:

|                         |                                 |
|-------------------------|---------------------------------|
| --episodes, -n N        | Number of training episodes     |
| --lr, --learning-rate F | Learning rate (default: 0.001)  |
| --batch-size N          | Batch size (default: 64)        |
| --gamma F               | Discount factor (default: 0.99) |

### Exploration:

|                       |                                 |
|-----------------------|---------------------------------|
| --exploration-start F | Starting epsilon (default: 1.0) |
| --exploration-end F   | Final epsilon (default: 0.05)   |
| --exploration-decay F | Decay rate (default: 0.9995)    |

### Network:

|                        |                                               |
|------------------------|-----------------------------------------------|
| --target-update-freq N | Target network update interval (default: 100) |
| --buffer-capacity N    | Replay buffer size (default: 100000)          |
| --min-buffer-size N    | Min samples before training (default: 1000)   |

### Rewards:

|                      |                                                   |
|----------------------|---------------------------------------------------|
| --win-gin-reward F   | Reward for gin (default: 50.0)                    |
| --win-knock-reward F | Reward for knock (default: 20.0)                  |
| --deadwood-bonus F   | Per-point deadwood reduction bonus (default: 0.1) |
| --meld-bonus F       | Meld completion bonus (default: 1.0)              |

### Output:

|                                |                                           |
|--------------------------------|-------------------------------------------|
| <code>--output, -o PATH</code> | Model save path (default: auto-generated) |
| <code>--name NAME</code>       | Experiment name                           |
| <code>--no-tensorboard</code>  | Disable TensorBoard logging               |

## Experiment Output

Each experiment automatically saves: - Model checkpoint: `models/experiments/<name>.pt` - Config file: `models/experiments/<name>.json` (for reproducibility) - TensorBoard logs: `runs/<name>/`

## Suggested Experiments

Start with these to find what works best for your setup:

*# Experiment 1: Lower learning rate (more stable)*

```
uv run gin-experiment --name "lr_0003" --lr 0.0003 --episodes 15000
```

*# Experiment 2: Larger batches (smoother gradients)*

```
uv run gin-experiment --name "batch_128" --batch-size 128 --episodes 15000
```

*# Experiment 3: Slower exploration (more exploitation time)*

```
uv run gin-experiment --name "slow_decay" --exploration-decay 0.9998 --episodes 20000
```

*# Experiment 4: Higher reward signals*

```
uv run gin-experiment --preset aggressive-rewards --name "high_rewards"
```

*# Experiment 5: Combined tweaks*

```
uv run gin-experiment --name "combined" --lr 0.0005 --batch-size 128 --gamma 0.95
```

## Comparing Results

Use TensorBoard to compare experiments:

```
tensorboard --logdir runs/
# Open http://localhost:6006
```

Or compare models directly:

*# Test each trained model*

```
uv run gin-simulate --ai1-type learning --ai1-model models/experiments/lr_0003.pt --ai2-type basic -n 2
```

```
uv run gin-simulate --ai1-type learning --ai1-model models/experiments/batch_128.pt --ai2-type basic -n 2
```

## Monitoring Training

### TensorBoard

*# In a separate terminal*

```
tensorboard --logdir runs/
```

*# Open http://localhost:6006 in browser*

Metrics tracked: - `reward/episode` - Total reward per episode - `exploration_rate` - Current epsilon value - `eval/win_rate` - Win rate against BasicAI (evaluated periodically)

### Console Output

Training prints progress every 100 episodes:

```
Episode 100: reward=12.3, exploration=0.951, buffer=1523
```

```
Episode 200: reward=-5.2, exploration=0.904, buffer=3102
```

```
...
Episode 500: win_rate=0.42, avg_points=45.2, exploration=0.779
```

## Configuration

Training hyperparameters are defined in `gin_rummy/learning/trainer.py` (TrainingConfig) and `gin_rummy/learning/rewards.py` (RewardConfig). You can override them via CLI with `gin-experiment`:

### Default Values

**Training Config** (`trainer.py:42-83`):

```
num_episodes = 10_000
batch_size = 64
learning_rate = 0.001
gamma = 0.99          # Discount factor

exploration_start = 1.0
exploration_end = 0.05
exploration_decay = 0.9995

target_update_freq = 100 # Sync target network every N episodes
buffer_capacity = 100_000
min_buffer_size = 1000   # Start training after N experiences
```

**Reward Config** (`rewards.py:16-44`):

```
win_by_gin = 50.0
win_by_knock = 20.0
win_by_undercut = 30.0
deadwood_reduction_bonus = 0.1
meld_completion_bonus = 1.0
discard_kept_bonus = 0.5
discard_wasted_penalty = -1.0
```

**Network Architecture** (`models.py`):

```
DrawNet:    hidden_sizes = [128, 64]
DiscardNet: hidden_sizes = [256, 128]
KnockNet:   hidden_sizes = [64, 32]
```

## Model Files

Checkpoints are saved to `models/` directory:

```
models/
+-- learning_ai.pt    # Contains all three networks + metadata
```

Metadata includes: - `episode` - Training episode when saved - `exploration_rate` - Current epsilon - `curriculum_idx` - Current curriculum stage

## Tips for Training

1. **Start with fewer episodes** to verify setup works:

```
uv run gin-train --episodes 100 --eval-freq 50
```

2. **Monitor early win rate** - should improve from ~30% to 50%+ against BasicAI within first 2000 episodes.

3. **Resume training** if interrupted:

```
uv run gin-train --resume models/learning_ai.pt --episodes 5000
```

4. **Compare to baseline** after training:

```
# Learning vs Context
```

```
uv run gin-simulate --ai1-type learning --ai1-model models/learning_ai.pt --ai2-type context -n 100
```

```
# Learning vs Basic
```

```
uv run gin-simulate --ai1-type learning --ai1-model models/learning_ai.pt --ai2-type basic -n 100
```

## Expected Results

After full training (20,000 episodes): - Win rate vs BasicAI: Target >60% - Win rate vs ContextAwareAI: Target >55%

Note: An untrained LearningAI (random networks) will lose nearly every game. Training is required before the model is competitive.